Revisión de Matemáticas en JavaScript

Trabajando con el Tipo de Dato Número

- **Definición**: El tipo `Number` de JavaScript incluye enteros, números de coma flotante, `Infinity` y `NaN`. Los números de coma flotante son números con un punto decimal. Positive `Infinity` is a number greater than any other number while `-Infinity` is a number smaller than any other number. `NaN` (`Not a Number`) represents an invalid numeric value like the string `"Jessica"`.

Operaciones Aritméticas Comunes

- **Operador de Suma**: Este operador (`+`) se utiliza para calcular la suma de dos o más números.
- **Operador de Resta**: Este operador (`-`) se utiliza para calcular la diferencia entre dos números.
- **Operador de Multiplicación**: Este operador (`*`) se utiliza para calcular el producto de dos o más números.
- **Operador de División**: Este operador (`/`) se utiliza para calcular el cociente entre dos números.
- **División por Cero**: Si intentas dividir por cero, JavaScript devolverá `Infinity`.
- **Operador de Resto**: Este operador (`%`) devuelve el resto de una división.
- **Operador de Exponenciación**: Este operador (`**`) eleva un número a la potencia de otro.

Cálculos con Números y Cadenas

- **Explicación**: Cuando usas el operador `+` con un número y una cadena, JavaScript convertirá el número en una cadena y concatenará los dos valores. Cuando usas los operadores `-`, `*` o `/` con una cadena y un número, JavaScript convertirá la cadena en un número y el resultado será un número. Para `null` y `undefined`, JavaScript trata `null` como 0 y `undefined` como `NaN` en operaciones matemáticas.

```

console.log(result); // "510"
console.log(typeof result); // string

const subtractionResult = '10' - 5;
console.log(subtractionResult); // 5
console.log(typeof subtractionResult); // number

const multiplicationResult = '10' * 2;
console.log(multiplicationResult); // 20
console.log(typeof multiplicationResult); // number

const divisionResult = '20' / 2;
console.log(divisionResult); // 10
console.log(typeof divisionResult); // number

const result1 = null + 5;
console.log(result1); // 5
console.log(typeof result1); // number

const result2 = undefined + 5;
```

```
5
"number"
20
"number"
10
"number"
```

Precedencia de Operadores

- **Definición**: La precedencia de operadores determina el orden en el que las operaciones se evalúan en una expresión. Los operadores con mayor precedencia se evalúan antes que los de menor precedencia. Los valores dentro del paréntesis se evaluarán primero y la multiplicación/división tendrá mayor precedencia que la suma/resta. Si los operadores tienen la misma precedencia, entonces JavaScript utilizará la asociatividad.

```
const result = (2 + 3) * 4;

console.log(result); // 20

const result2 = 10 - 2 + 3;

console.log(result2); // 11

const result3 = 2 ** 3 ** 2;

console.log(result3); // 512
```

```
20
11
512
```

- **Definición**: La asociatividad nos informa la dirección en la que se evalúa una expresión cuando existen múltiples operadores del mismo tipo. Define si la expresión debe evaluarse de izquierda a derecha (`asociatividad a la izquierda`) o de derecha a izquierda (`asociatividad a la derecha`). Por ejemplo, el operador de exponenciación también es asociado de derecha a izquierda:

```
const result4 = 5 ** 4 ** 1;

console.log(result4); // 625
```

freeCodeCamp(🔥)

Certificación de JavaScript | Revisión de Matemáticas en JavaScript

```
625
```

Operadores de Incremento y Decremento

- **Operador de Incremento**: Este operador se utiliza para aumentar el valor en uno. La notación prefija `++num` incrementa el valor de la variable primero, luego devuelve un nuevo valor. La notación postfija `num++` devuelve el valor actual de la variable primero, luego la incrementa.

```
let x = 5;

console.log(++x); // 6
console.log(x); // 6


let y = 5;

console.log(y++); // 5
console.log(y); // 6
```

```
6
```

```
6
```

```
5
```

```
6
```

freeCodeCamp

Certificación de JavaScript | Revisión de Matemáticas en JavaScript

```
console.log(--num); // 4
console.log(num--); // 4
console.log(num); // 3
```

```
4
4
3
```

Operadores de Asignación Compuesta

- **Operador de Asignación de Suma (`+=`)**: Este operador realiza la suma sobre los valores y asigna el resultado a la variable.
- **Operador de Asignación de Resta (`-=`)**: Este operador realiza la resta sobre los valores y asigna el resultado a la variable.
- **Operador de Asignación de Multiplicación (`*=`)**: Este operador realiza la multiplicación sobre los valores y asigna el resultado a la variable.
- **Operador de Asignación de División (`/=`)**: Este operador realiza la división sobre los valores y asigna el resultado a la variable.
- **Operador de Asignación de Resto (`%=`)**: Este operador divide una variable por el número especificado y asigna el resto a la variable.
- **Operador de Asignación de Exponenciación (`**=`)**: Este operador eleva una variable a la potencia del número especificado y reasigna el resultado a la variable.

Booleanos y Igualdad

- **Definición de Booleano**: Un booleano es un tipo de dato que solo puede tener dos valores: `true` o `false`.
- **Operador de Igualdad (`==`)**: Este operador utiliza la coerción de tipo antes de verificar si los valores son iguales.

```
console.log(5 == '5'); // true
```

```
true
```

- **Operador de Estricta Igualdad** (`===`): Este operador no realiza coerción de tipos y verifica si ambos, tanto los tipo como los valores, son iguales.

```
console.log(5 === '5'); // false
```

```
false
```

Operadores de Comparación

- **Operador Mayor Que (`>`)**: Este operador verifica si el valor a la izquierda es mayor que el de la derecha.
- **Operador Mayor o Igual (`>=`)**: Este operador verifica si el valor a la izquierda es mayor o igual que el de la derecha.
- **Operador Menor Que (`<`)**: Este operador verifica si el valor a la izquierda es menor que el de la derecha.
- **Operador Menor Que (`<=`) o Igual**: Este operador verifica si el valor a la izquierda es menor o igual que el de la derecha.

Operadores Unarios

- **Operador Más Unario**: Este operador convierte a su operando en un número. Si el operando ya es un número, permanece sin cambios.

```
const str = '42';
const num = +str;

console.log(num); // 42
console.log(typeof num); // number
```

```
42
"number"
```

- **Operador Negación Unaria (`-`)**: Este operador niega el operando.

```
const num = 4;
console.log(-num); // -4
```

```
-4
```

- **Operador Lógico NO (`!`)**: Este operador invierte el valor booleano de su operando. Por lo tanto, si el operando es `true`, se convierte en `false`, y si es `false`, se convierte en `true`.

Operadores Bit a Bit

- **Operador AND a nivel de bits (`&`)**: Este operador devuelve un 1 en cada posición de bit para la cual los bits correspondientes de ambos operandos son 1.
- **Operador de Asignación AND a Nivel de Bits (`&=`)**: Este operador realiza una operación de `bitwise AND` con el número especificado y reasigna el resultado a la variable.
- **Operador O Bit a Bit (`|`)**: Este operador devuelve un 1 en cada posición de bit para la cual los bits correspondientes de cualquiera o ambos operandos son 1.
- **Asignación OR a Nivel de Bit (Operador `|=`)**: Este operador realiza una operación `OR a nivel de bit` con el número especificado y reasigna el resultado a la variable.
- **Operador XOR a Nivel de Bit (`^`)**: Este operador devuelve un 1 en cada posición de bit para la cual los bits correspondientes de cualquiera de los operandos son 1, pero no ambos.
- **Operador NOT a Nivel de Bit (`~`)**: Este operador invierte la representación binaria de un número.
- **Operador de desplazamiento a la izquierda (`<<`)**: Este operador desplaza todos los bits hacia la izquierda un número especificado de posiciones.
- **Operador de desplazamiento a la derecha (`>>`)**: Este operador desplaza todos los bits hacia la derecha.

Condicionales, Valores Verídicos, Valores Falsos y el Operador Ternario

- **`if/else if/else`**: Una declaración `if` toma una condición y ejecuta un bloque de código si esa condición es `verdadera`. Si la condición es `false`, entonces se mueve al bloque `else if`. If none of those conditions are `true`, then it will execute the `else` clause. `Truthy` values are any values that result in `true` when evaluated in a Boolean context like an `if` statement. `Falsy` values are values that evaluate to `false` in a Boolean context.

```
const score = 87;

if (score >= 90) {
 console.log('You got an A');
} else if (score >= 80) {
 console.log('You got a B'); // You got a B
} else if (score >= 70) {
 console.log('You got a C');
} else {
 console.log('You failed! You need to study more!');
}
```

```
"You got a B"
```

- **Operador Ternario**: Este operador se utiliza frecuentemente como una manera más corta de escribir declaraciones `if else`.

```
const temperature = 30;
const weather = temperature > 25 ? 'sunny' : 'cool';

console.log(`It's a ${weather} day!`); // It's a sunny day!
```

```
"It's a sunny day!"
```

Operadores Lógicos Binarios

freeCodeCamp(🔥)

Certificación de JavaScript | Revisión de Matemáticas en JavaScript

```
console.log(result); // hello
```

```
"hello"
```

- **Operador OR lógico (`||`)**: Este operador verifica si al menos uno de los operandos es verdadero.
- **Operador de Coalescencia Nula (`??`)**: Este operador devolverá un valor solo si el primero es `null` o `undefined`.

```
const userSettings = {
 theme: null,
 volume: 0,
 notifications: false,
};

let theme = userSettings.theme ?? 'light';
console.log(theme); // light
```

El Objeto `Math`

- **El Método** `Math.random()`: Este método genera un número flotante aleatorio entre 0 (inclusivo) y 1 (exclusivo). Esto significa que el resultado posible puede ser 0, pero nunca llegará a 1.
- **El método** `Math.max()`: Este método toma un conjunto de números y devuelve el valor máximo.
- **El método** `Math.min()`: Este método toma un conjunto de números y devuelve el valor mínimo.
- **El Método** `Math.ceil()`: Este método redondea un valor hacia arriba al número entero más cercano.
- **El Método** `Math.floor()`: Este método redondea un valor hacia abajo hasta el número entero más cercano.
- **El Método** `Math.round()`: Este método redondea un valor al número entero más cercano.

```
console.log(Math.round(2.3)); // 2
console.log(Math.round(4.5)); // 5
console.log(Math.round(4.8)); // 5
```

```
2
5
5
```

- **El Método** `Math.trunc()`: Este método elimina la parte decimal de un número, devolviendo solo la parte entera, sin redondear.
- **El Método** `Math.sqrt()`: Este método devolverá la raíz cuadrada de un número.
- **El Método** `Math.cbrt()`: Este método devolverá la raíz cúbica de un número.

está indefinido. The `isNaN()` function property is used to determine whether a value is `NaN` or not. `Number.isNaN()` provides a more reliable way to check for `NaN` values, especially in cases where type coercion might lead to unexpected results with the global `isNaN()` function.

```
console.log(isNaN(NaN));       // true
console.log(isNaN(undefined)); // true
console.log(isNaN({}));        // true

console.log(isNaN(true));      // false
console.log(isNaN(null));      // false
console.log(isNaN(37));        // false


console.log(Number.isNaN(NaN));        // true
console.log(Number.isNaN(Number.NaN)); // true
console.log(Number.isNaN(0 / 0));      // true

console.log(Number.isNaN("NaN"));      // false
console.log(Number.isNaN(undefined));  // false
```

```
true
true
false
false
false
true
true
true
false
```

- **El Método `parseFloat()`**: Este método analiza una cadena y devuelve un número de punto flotante. Está diseñado para extraer un número del comienzo de una cadena, incluso si la cadena contiene caracteres no numéricos más adelante.
- **The `parseInt()` Method**: This method parses a string argument and returns an integer. `parseInt()` stops parsing at the first non-digit it encounters. Para números de punto flotante, solo devuelve la parte entera. Si no puede encontrar un entero válido al inicio de la cadena, devuelve `NaN`.
- **El Método `toFixed()`**: Este método se llama en un número y toma un argumento opcional, que es el número de dígitos que aparecerán después del punto decimal. Devuelve una representación en cadena del número con el número especificado de decimales.

Asigna

- [ ] Revise los temas y conceptos matemáticos de JavaScript.

Por favor, complete la asignación

freeCodeCamp(🔥)

Certificación de JavaScript | Revisión de Matemáticas en JavaScript